Ridge Regression

Study Material · Theory + Code, block by block

Regularised linear regression with scikit-learn

DIGITAL GYAAN

THEORY · 1

What is Ridge Regression?

Ridge is **linear regression with a penalty**. It fits the usual best-fit line, but adds a term that keeps the feature weights **small and stable**.

- It uses an **L2 penalty** (the sum of squared weights).
- Weights are shrunk **toward** zero — but never **exactly** zero, so **all features stay**.
- Goal: reduce **over-fitting** and stabilise the model.

THEORY · 2

The problem it solves

Ordinary least squares can **over-fit** when there are many features or when features are **correlated** with each other.

- Correlated features make weights **large and unstable** (high variance).
- Tiny changes in data → wildly different coefficients.
- Ridge's penalty **shrinks** those weights, trading a little bias for a big drop in variance.
- The model then **generalises better** to new data.

THEORY · 3

The maths

Ridge picks the weights **w** that minimise:

$$\text{Cost} = \Sigma\,(y_i - \hat{y}_i)^2 \;+\; \alpha\,\Sigma\, w_j^2$$

- $\Sigma(y_i - \hat{y}_i)^2$ = residual sum of squares (how well we fit the data).
- $\alpha\,\Sigma\, w_j^2$ = the L2 penalty (how big the weights are).

THEORY · 4

The role of alpha (α)

Alpha sets how strong the penalty is — the dial between fitting the data and keeping weights small.

- α **= 0** → no penalty — plain linear regression.
- **Small** α → light shrinkage, weights stay close to OLS.
- **Large** α → strong shrinkage, weights pushed hard toward 0.
- We don't guess it — we **test several values** and let cross-validation choose.

THEORY · 5

Key properties to remember

- **Keeps all features** — it shrinks, it never drops (that's Lasso's job).
- **Handles multicollinearity** by spreading weight across correlated features.
- **Bias–variance trade-off:** adds a little bias, removes a lot of variance.
- **Scale matters** — features on similar scales make the penalty fair (scaling is good practice).
- Has a clean **closed-form solution**, so it trains fast.

THEORY · 6

When should you use Ridge?

Reach for Ridge when you have **many predictors that may be correlated** and you believe **most of them matter**.

Strengths

- Stable, reliable coefficients
- Great with correlated features
- Fast — closed-form solution

Trade-offs

- Keeps every feature (no selection)
- Needs alpha tuning
- Want sparsity? Use Lasso instead

Code Walkthrough

The Ridge script, explained one block at a time

CODE · BLOCK 1 OF 6

Import the libraries

```
import pandas as pd
import numpy as np
from sklearn.datasets import fetch_openml
```

What this block does

- **pandas** — handles the data as a table (DataFrame).
- **numpy** — fast numerical / array math.
- **fetch_openml** — downloads ready-made datasets from openml.org.

CODE · BLOCK 2 OF 6

Load the dataset

```
boston = fetch_openml(name="boston",
        version=1, as_frame=True,
        parser="pandas")
df = boston.frame
df["Price"] = boston.target
```

What this block does

- Downloads the **Boston housing** dataset.
- `df` is the table of features (rooms, crime rate…).
- We add the value we want to predict as a column called **Price**.

CODE · BLOCK 3 OF 6

Split inputs (X) and answer (Y)

```
X = df.drop(columns=["MEDV", "Price"])
Y = df["Price"]

X = X.astype(float)
Y = Y.astype(float)
```

What this block does

- **X** = the features the model learns from.
- **Y** = the answer (the price).
- We drop **both** price columns from X so the model can't “cheat”.
- Cast everything to **float** so the maths works.

CODE · BLOCK 4 OF 6

Build the model + list alphas

```
from sklearn.linear_model import Ridge
from sklearn.model_selection import GridSearchCV

ridge = Ridge()
params = {"alpha": [1e-15, 1e-10, 1e-8,
          1e-3, 1e-2, 1, 5, 10, 20]}
```

What this block does

- Create a **Ridge** model with default settings.
- `params` lists every **alpha** we want to try — from almost 0 up to 20.
- Bigger alpha = stronger shrinkage.

Search + cross-validate

```
ridge_cv = GridSearchCV(ridge, params,
    scoring="neg_mean_squared_error",
    cv=5)
ridge_cv.fit(X, Y)
```

What this block does

- **GridSearchCV** trains the model once for **each** alpha.
- **cv=5** — splits data into 5 parts, trains on 4 and tests on 1, five times.
- **scoring** — lower error is better (negative, so closer to 0 wins).
- `.fit()` runs the whole search.

CODE · BLOCK 6 OF 6

Show the results

```
print(ridge_cv.best_params_)
print(ridge_cv.best_score_)
```

OUTPUT

```
{'alpha': 20}
-32.38025025182516
```

Reading the result

- **best_params_** → the winning alpha (here **20**).
- **best_score_** → the negative MSE (closer to 0 = better).
- Score ≈ –32.4 → typical error ≈ √32.4 ≈ 5.7, about **$5,700** per house.

REFERENCE

The full script

```
import pandas as pd
from sklearn.datasets import fetch_openml
from sklearn.linear_model import Ridge
from sklearn.model_selection import GridSearchCV

boston = fetch_openml(name="boston", version=1, as_frame=True, parser="pandas")
df = boston.frame
df["Price"] = boston.target

X = df.drop(columns=["MEDV", "Price"]).astype(float)
Y = df["Price"].astype(float)

ridge = Ridge()
params = {"alpha": [1e-15, 1e-10, 1e-8, 1e-3, 1e-2, 1, 5, 10, 20]}

ridge_cv = GridSearchCV(ridge, params, scoring="neg_mean_squared_error", cv=5)
ridge_cv.fit(X, Y)

print(ridge_cv.best_params_)   # {'alpha': 20}
print(ridge_cv.best_score_)    # -32.380...
```

RECAP

Ridge in one minute

- Ridge = linear regression **+ L2 penalty** ($\alpha \Sigma w^2$).
- It **shrinks** weights toward zero but **keeps every feature**.
- **Alpha** controls the shrinkage; pick it with **cross-validation**.
- Best for **many, correlated features** where you want stability.
- On Boston data the best alpha was **20**, MSE ≈ **32.4**.